ASR Tutorial with Kaldi

From Fundamentals to Practical Implementation

Dr. Ye Kyaw Thu

August 28, 2026

Lab Leader, Language Understanding Lab, AI Mario, Myanmar
Visiting Professor, Core-AI Research Team, NECTEC, Thailand

Outline

Introduction to Kaldi

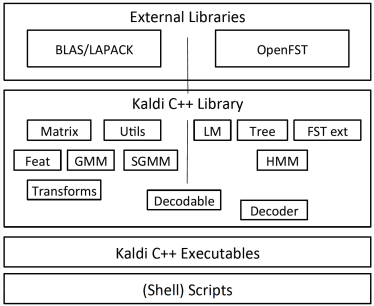
What is Kaldi?

- Kaldi is an open-source toolkit for speech recognition and related applications.
- Developed primarily in C++, with scripts in Bash, Python, and Perl.
- Licensed under Apache 2.0, making it free for academic and commercial use.
- Designed for research: provides extreme flexibility and modularity.

Why Use Kaldi?

- **State-of-the-art:** Implements TDNNs, TDNN-Fs, LSTMs, and Chain models.
- **Reproducible Recipes:** Complete scripts for databases like LibriSpeech, WSJ, AN4.
- **WFST Framework:** Uses OpenFst for efficient graph composition.
- **Community:** Widely adopted in academia and industry since 2011.

Kaldi Architecture Overview



A simplified view of the different components of Kaldi.

(Figure from Daniel Povey et al., 2011)

- The toolkit integrates OpenFst for graph operations and BLAS/LAPACK for linear algebra.
- The `DecodableInterface` bridges the acoustic models (GMM/DNN) and the FST decoders.
- This modular design allows easy extension and swapping of components.

Kaldi vs. End-to-End Models

Kaldi (Hybrid HMM-DNN)

- Requires separate Acoustic and Language models.
- Uses complex FST graphs for decoding.
- Highly optimized for low-latency decoding.

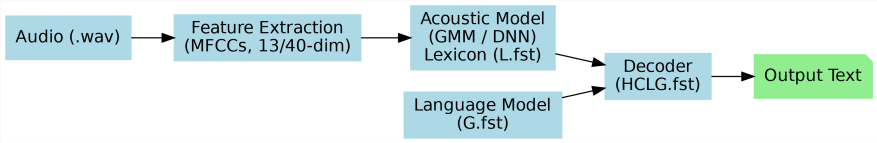
End-to-End (E2E)

- Maps audio directly to text (e.g., RNN-T, CTC).
- Simpler pipeline, but harder to interpret.
- Harder to integrate external Language Models.

Kaldi remains a standard for benchmarking and production ASR.

ASR System Overview

The ASR Pipeline



- **Feature Extraction:** Audio → MFCCs.
- **Acoustic Model:** MFCCs → Phoneme probabilities.
- **Lexicon & Language Model:** Words → Phonemes & Word probabilities.
- **Decoder:** Combines all components to output the most likely text.

Feature Extraction (MFCCs)

- Audio is converted into a sequence of feature vectors (typically 10ms frame shift).
- **MFCCs (Mel-Frequency Cepstral Coefficients):**
  - Mimic human auditory perception (Mel scale).
  - Standard dimension is 13.
  - Often appended with Delta (Δ) and Double-Delta ($\Delta\Delta$) to capture temporal dynamics $\rightarrow$ 39 dims.
- **CMVN:** Cepstral Mean Variance Normalization is applied to reduce channel effects.

Acoustic Modeling (GMM vs DNN)

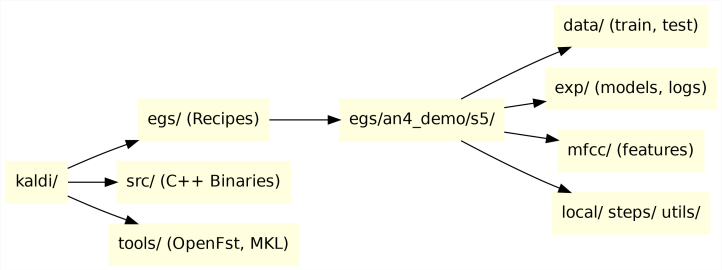
- Maps feature vectors to HMM states (Senones).
- **GMM (Gaussian Mixture Model):**
  - Classical approach, models probability density of features.
  - Used for bootstrapping and alignment.
- **DNN (Deep Neural Network):**
  - Predicts posterior probabilities of senones.
  - Requires alignments from GMMs for training (Cross-Entropy).
  - Kaldi's `nnet3` framework supports TDNNs, LSTMs, etc.

Language Modeling & Lexicon

- **Lexicon (L.fst):** Maps words to phoneme sequences.
  - Example: "HELLO" $\rightarrow$ HH AH L OW
- **Language Model (G.fst):** Probabilities of word sequences.
  - N-grams (e.g., 3-gram) are standard.
  - Restricts the search space to valid word combinations.
- Kaldi combines these into a single decoding graph: **HCLG.fst**.

Kaldi Folder Structures & Files

Kaldi Directory Structure



- `egs/`: Recipes for various datasets.
- `src/`: Compiled C++ binaries (e.g., `featbin`, `gmmbin`).
- `tools/`: Dependencies (OpenFst, ATLAS/MKL).

Inside an egs/<recipe>/s5/ Directory

- `run.sh`: The main script that runs the entire pipeline.
- `cmd.sh`: Defines execution backend (e.g., `run.pl` for local, `slurm.pl` for cluster).
- `path.sh`: Sets up environment variables and `PATH`.
- `conf/`: Configuration files (e.g., MFCC configs).
- `local/`: Dataset-specific preparation scripts.
- `steps/ & utils/`: Standard, dataset-agnostic Kaldi scripts.

The `data/` Directory

Kaldi requires specific text files for training and testing:

- `wav.scp`: Maps utterance IDs to audio file paths or commands.
- `text`: Maps utterance IDs to their transcriptions.
- `utt2spk`: Maps utterance IDs to speaker IDs.
- `spk2utt`: Inverse of `utt2spk`.
- `feats.scp`: Points to extracted feature matrices.
- `cmvn.scp`: Points to Cepstral Mean Variance Normalization matrices.

All files must be sorted! Kaldi scripts rely on strict Unix sorting.

Example: wav.scp and text

wav.scp

```
fcaw-an406-b sph2pipe -f wav -p -c 1 data/an4/wav/an4test_clstk/fcaw/
    an406-fcaw-b.sph |
```

text

```
fcaw-an406-b RUBOUT G M E F THREE NINE
```

Note the pipe | at the end of the wav.scp line: Kaldi executes this as a command to stream audio, allowing on-the-fly conversion from .sph to .wav.

The exp/ Directory

- Contains all experimental outputs: models, logs, alignments.
- exp/mono0a/: Monophone GMM model.
- exp/tri1/: Triphone GMM model.
- exp/tri1_ali/: Alignments generated using Triphone model.
- exp/nnet3/: Neural network models.
- exp/<model>/decode/: Decoding results and WER files.

Installation & Setup

Installing Kaldi

1. Clone the repository:

```
git clone https://github.com/kaldi-asr/kaldi.git
```

2. Install tools (OpenFst, BLAS):

```
cd kaldi/tools
make -j 8
extras/install_mkl.sh
```

3. Configure and compile the source:

```
cd ../src
./configure --shared --use-cuda=yes
make depend -j 8
make -j 8
```

Common Installation Issues

- **Missing MKL/OpenBLAS:** Linear algebra libraries are required. `install_mkl.sh` usually fixes this.
- **CUDA Compatibility:** Ensure your GPU driver matches the CUDA toolkit. Kaldi's `configure` script will fail if it can't find `nvcc`.
- **GCC Version:** Newer GCC (v13+) may break older C++ code. Kaldi compiles best with GCC 9-11.
- **CUB/Thrust Conflicts:** CUDA 12+ may require modifying `kaldi.mk` to ignore CUB version checks.

Training Overview & Feature Extraction

The ASR Training Pipeline

1. **Data Preparation:** Create `wav.scp`, `text`, etc.
2. **Lang Preparation:** Build Lexicon (L.fst) and Language Model (G.fst).
3. **Feature Extraction:** Compute MFCCs and CMVN.
4. **GMM Training:** Monophone $\rightarrow$ Triphone (for alignment).
5. **DNN Training:** Train neural network using GMM alignments.
6. **Decoding & Evaluation:** Build HCLG graph and test.

Feature Extraction in Kaldi

```
steps/make_mfcc.sh --nj 8 data/train exp/make_mfcc/train mfcc
```

- `--nj 8`: Number of parallel jobs.
- `data/train`: Input data directory.
- `exp/make_mfcc/train`: Log directory.
- `mfcc`: Output directory for `.ark` and `.scp` files.

This generates 13-dimensional raw MFCCs. Kaldi GMM scripts automatically append Deltas to make 39 dims.

Training Acoustic Models

Step 1: Monophone Training

```
steps/train_mono.sh --nj 4 --cmd "utils/run.pl" \
  data/train data/lang exp/mono0a
```

- Trains a context-independent GMM-HMM.
- Initializes a flat-start topology and iteratively aligns data.
- Fast, but not highly accurate. Used as a starting point.

Step 2: Triphone Training

```
steps/train_deltas.sh --cmd "utils/run.pl" \
  300 3000 data/train data/lang exp/mono0a_ali exp/tri1
```

- Trains a context-dependent (Triphone) GMM-HMM.
- Uses decision trees to tie similar phonetic contexts.
- 300: Number of leaves (states).
- 3000: Number of Gaussians.

From GMMs to DNNs: The Alignment Problem

- Neural Networks cannot be trained from scratch on raw audio easily.
- We need **frame-level alignments** (which phoneme occurs at which 10ms frame).
- We use the Triphone GMM to align the training data:

```
steps/align_si.sh --nj 4 data/train data/lang exp/tri1 exp/tri1_ali
```

- These alignments are the "ground truth" labels for DNN training (Cross-Entropy objective).

Step 3: DNN (nnet3) Training

```
steps/nnet3/train_dnn.py --use-gpu true \
  --feat-dir=data/train --ali-dir=exp/tri1_ali \
  --dir=exp/nnet3/dnn1
```

- `nnet3` is Kaldi's modern neural network framework.
- Supports TDNNs, LSTMs, and CNNs.
- Configuration is defined via `xconfig` files.
- Automatically handles LDA, preconditioning, and natural gradient.

Decoding & Evaluation

Building the Decoding Graph (HCLG.fst)

```
utils/mkgraph.sh data/lang exp/tri1 exp/tri1/graph
```

- Combines Context (C), Lexicon (L), and Language Model (G) with the HMM (H).
- Result: `HCLG.fst`.
- The decoder searches this graph for the best path given the audio features.

Decoding the Test Set

```
steps/decode.sh --nj 4 exp/tri1/graph data/test exp/tri1/decode
```

- Generates lattices containing multiple hypotheses.
- Runs scoring script to compute Word Error Rate (WER).

Word Error Rate (WER) is the standard ASR metric:

$$WER = \frac{Substitutions + Deletions + Insertions}{TotalWords}$$

Practical Exercise of ASR

Tutorial Overview: 3 Jupyter Notebooks

1. **`yesno_demo`:** The "Hello World" of Kaldi.
   - Tiny dataset (1 hour), runs in 2 minutes on CPU.
   - Complete Monophone GMM pipeline.
2. **`fsdd_demo`:** Free Spoken Digit Dataset.
   - Custom data preparation (parsing GitHub dataset).
   - Monophone vs. Triphone comparison.
3. **`an4_demo`:** From GMMs to DNNs.
   - Standard Kaldi recipe.
   - Feature dimensionality comparison (13-dim vs 40-dim).
   - Neural Network (nnet3) training on GPU.

Notebook 1: `yesno_demo`

- Goal: Understand the Kaldi pipeline flow without worrying about complex data.
- Data: People saying "yes" or "no".
- Steps executed in the notebook:
  - `local/prepare_data.sh`
  - `utils/prepare_lang.sh`
  - `steps/make_mfcc.sh`
  - `steps/train_mono.sh`
  - `steps/decode.sh`
- Result: 0% WER (perfect recognition).

Notebook 2: `fsdd_demo` (Digits)

- Goal: Learn how to build a custom dictionary and Language Model from scratch.
- Data: 6 speakers saying digits 0-9.
- Key challenge: Parsing filenames to create Kaldi `text` and `wav.scp` files.
- Results:
  - Monophone WER: 14.10%
  - Triphone WER: 17.90% (Overfitting due to small dataset!)
- Lesson: Complex models (Triphones) need more data to generalize.

Notebook 3: `an4_demo` (GMM vs DNN)

- Goal: Train a DNN using Kaldi's `nnet3` framework.
- Data: AN4 (spelled letters and numbers).
- Experiment 1: DNN with 13-dim raw MFCCs.
- Experiment 2: DNN with 40-dim high-resolution MFCCs.

Notebook 3: `an4_demo` Results

Model	WER (%)
Monophone GMM (39-dim deltas)	11.25
Triphone GMM (39-dim deltas)	9.06
DNN (13-dim raw MFCCs)	23.67
DNN (40-dim hires MFCCs)	23.16

Why did the DNN perform worse?

- GMMs automatically use Deltas (temporal context).
- Our simple DNN looked at isolated 10ms frames without temporal context (splicing).
- Modern TDNNs solve this by splicing frames in the input layer.

Conclusion

Conclusion

- Kaldi provides a robust, modular framework for ASR research.
- The `data/`, `lang/`, and `exp/` directories form the core of any Kaldi recipe.
- GMM-HMMs remain essential for generating alignments to bootstrap Neural Networks.
- Feature engineering (MFCCs, Deltas, Hires) significantly impacts model performance.
- Practical debugging (sorting files, handling missing `<UNK>` symbols) is a critical skill.

Beyond Kaldi: Further Studies

- **Advanced Kaldi Topics:**
  - TDNNs with context splicing (e.g., `Append(-2,-1,0,1,2)`).
  - i-Vectors for speaker adaptation.
  - Chain Models using CTC objective for faster decoding.
- **Modern End-to-End (E2E) Frameworks:**
  - **ESPNet:** End-to-End speech processing toolkit (Transformer/Conformer).
  - **wav2vec 2.0:** Self-supervised learning for speech representations (HuggingFace).
  - **Whisper:** OpenAI's robust multilingual ASR model, highly popular for fine-tuning.

E2E Models for Low-Resource Languages (Our Research)

Applying modern ASR to local contexts: Fine-tuning Whisper and Transformers for Burmese.

- **iSAI-NLP 2025 (Phuket, Thailand):**
  - Ye Bhone Lin, Thura Aung, Ye Kyaw Thu, Thazin Myint Oo.
  - *"ASR Error Correction in Low-Resource Burmese with Alignment-Enhanced Transformers using Phonetic Features"*
  - Preprint: `https://arxiv.org/abs/2511.21088`
- **ICNLSP 2026 (Trento, Italy):**
  - Ye Kyaw Thu, Ye Bhone Lin, Thura Aung, et al.
  - *"myMediWhisper: Construction of Burmese Medical Speech Corpus and Whisper Fine-Tuning for Clinical Dialogue ASR"*
  - Preprint: `https://arxiv.org/abs/2608.11036`

References

References (1/2)

- Povey, D., et al. (2011). *The Kaldi Speech Recognition Toolkit*. IEEE ASRU.
- Kaldi Homepage: `https://kaldi-asr.org/`
- Kaldi GitHub Repository: `https://github.com/kaldi-asr/kaldi`
- Kaldi Documentation & Examples: `https://kaldi-asr.org/doc/examples.html`
- Eleanor Chodroff's Kaldi Tutorial: `https://eleanorchodroff.com/tutorial/kaldi/`
- JHU SS Kaldi Tutorial (Boulianne et al.): Provided PDF
- Linke, J., et al. (2023). *Using Kaldi for ASR of Conversational Austrian German*. `https://arxiv.org/abs/2301.06475`
- Hong, M., & Jiang, D. (2025). *A Practical Guide to Kaldi ASR Optimization*. `https://arxiv.org/abs/2506.07149v1`
- Free Spoken Digit Dataset (FSDD): `https://github.com/Jakobovski/free-spoken-digit-dataset`
- Kaldi Yes/No Tutorial: `https://github.com/keighrim/kaldi-yesno-tutorial`

References (2/2)

- **Modern E2E Frameworks:**
- ESPNet: `https://github.com/espnet/espnet`
- wav2vec 2.0: `https://huggingface.co/docs/transformers/model_doc/wav2vec2`
- Whisper: `https://openai.com/index/whisper/`
- **Low-Resource Burmese ASR Research:**
- Lin, Y.B., et al. (2025). *ASR Error Correction....* iSAI-NLP 2025. `https://arxiv.org/abs/2511.21088`
- Ye Kyaw Thu, et al. (2026). *myMediWhisper....* ICNLSP 2026. `https://arxiv.org/abs/2608.11036`

Questions?

Thank you for attending!